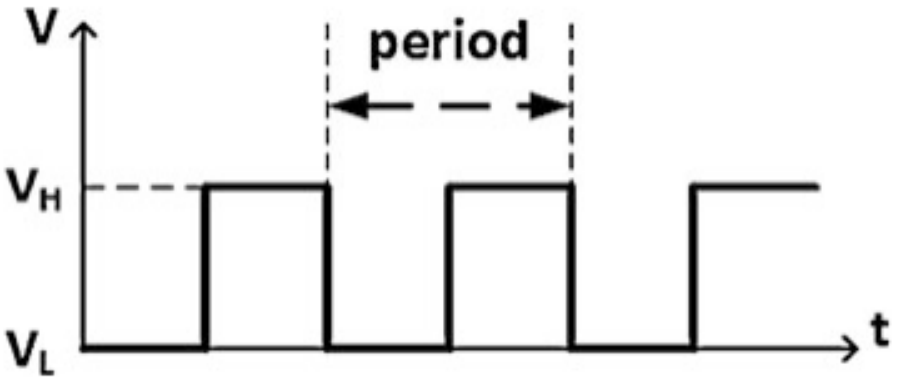
ee2004

Weeks 9, 10, 11

Timing Operations

- In order to understand time-based operations, we should first understand the clock signal and oscillator used to generate it.
- Clock is basically a periodic signal oscillating between high and low states with predefined frequency (specifically called clock frequency). A sample clock signal is given in Figure.
- Clock frequency has utmost importance since it defines the speed of operations within the embedded system.

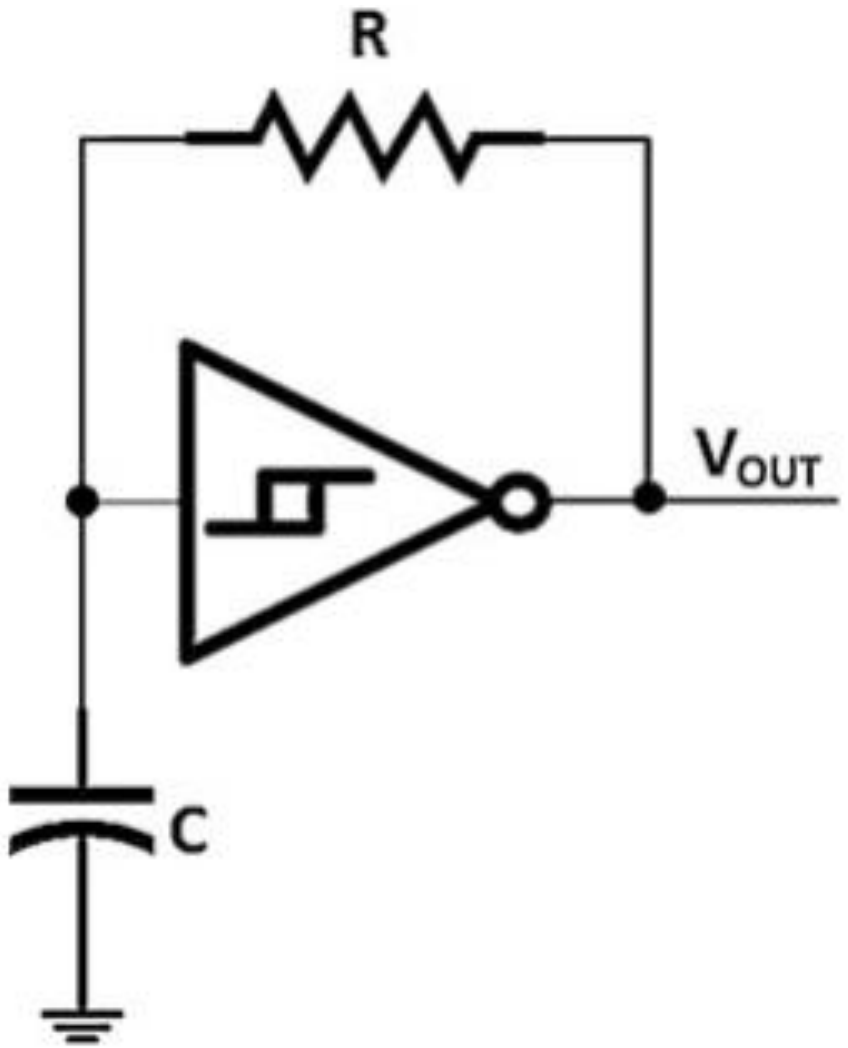


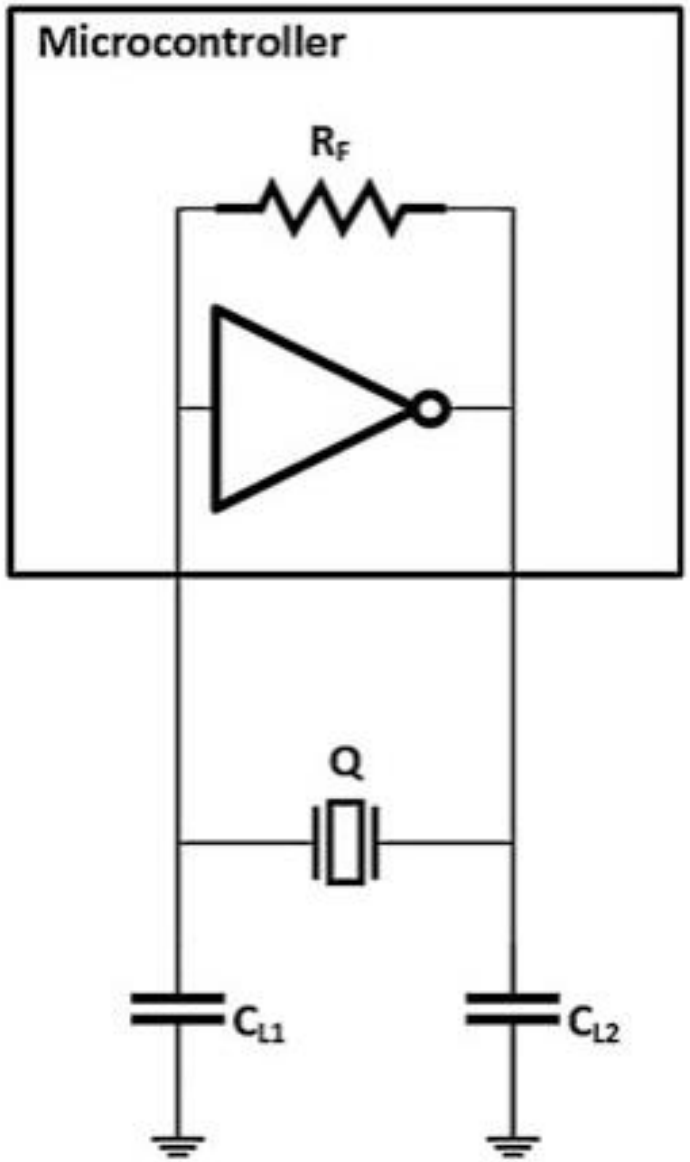
- Why does an embedded system (such as microcontroller) need a clock signal? In order to answer this question properly, we should understand the basic structure of an embedded system.
- Let’s focus on a microcontroller consisting of large number of logic gates. Through these, data transfer and basic operations are performed.
- A logic gate is formed by transistors with limited operation speed. Hence, there are times in which the logic gate is in undefined state such that it neither represents logic level 0 nor 1.
- A synchronization signal is needed to avoid these undefined states. Clock signal is used for this purpose such that at its each rising (or falling) edge, operations are performed.
- No operation is done between rising (or falling) edges of the clock signal.
- Hence, undefined states in logic gates are avoided. As a result, all operations within the microcontroller are done synchronously.

Oscillator as the Clock Signal Source

- Oscillator is an electronic circuit producing a periodic square wave which is the clock signal for our case.
- Therefore, an embedded system that needs a clock signal should have an oscillator or oscillators.
- An oscillator can be described by its two properties as accuracy and response time.
- Accuracy indicates how precise is the clock signal frequency.
- Response time indicates the time needed for the oscillator to stabilize such that it can generate the clock signal with the desired frequency.

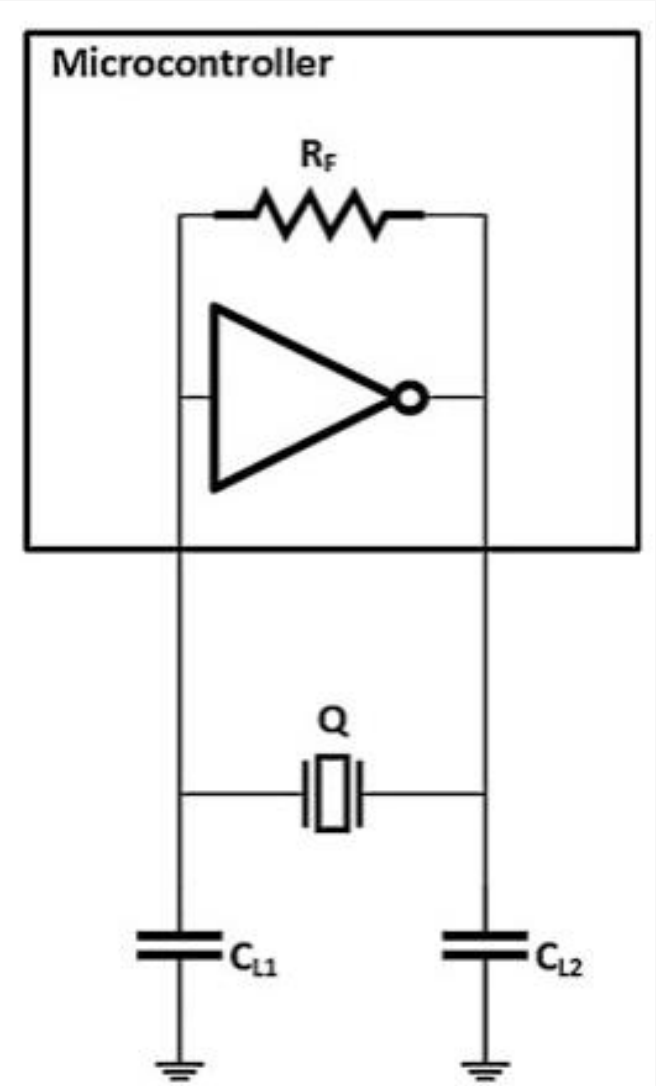
- There are mainly two oscillator types as resistive-capacitive (RC) and crystal. A basic Schmitt trigger RC oscillator is given in Fig.
- Here, frequency of the clock signal is determined by the R and C component values.
- Working principle of the RC oscillator is briefly as follows. The Schmitt trigger output goes to high level when the capacitor is charged. It goes to low level when the capacitor is discharged. Hence, the desired periodic clock signal can be produced.
- RC oscillators are mainly used as internal oscillators in microcontrollers since their components can be added to the hardware production process. Therefore, there is no additional component needed while using them.
- Besides, RC oscillators are cheap and have fast response time. However, their accuracy is low because of the passive components used in constructing the oscillator. Moreover, the accuracy can be affected from changing temperature and supply voltage values





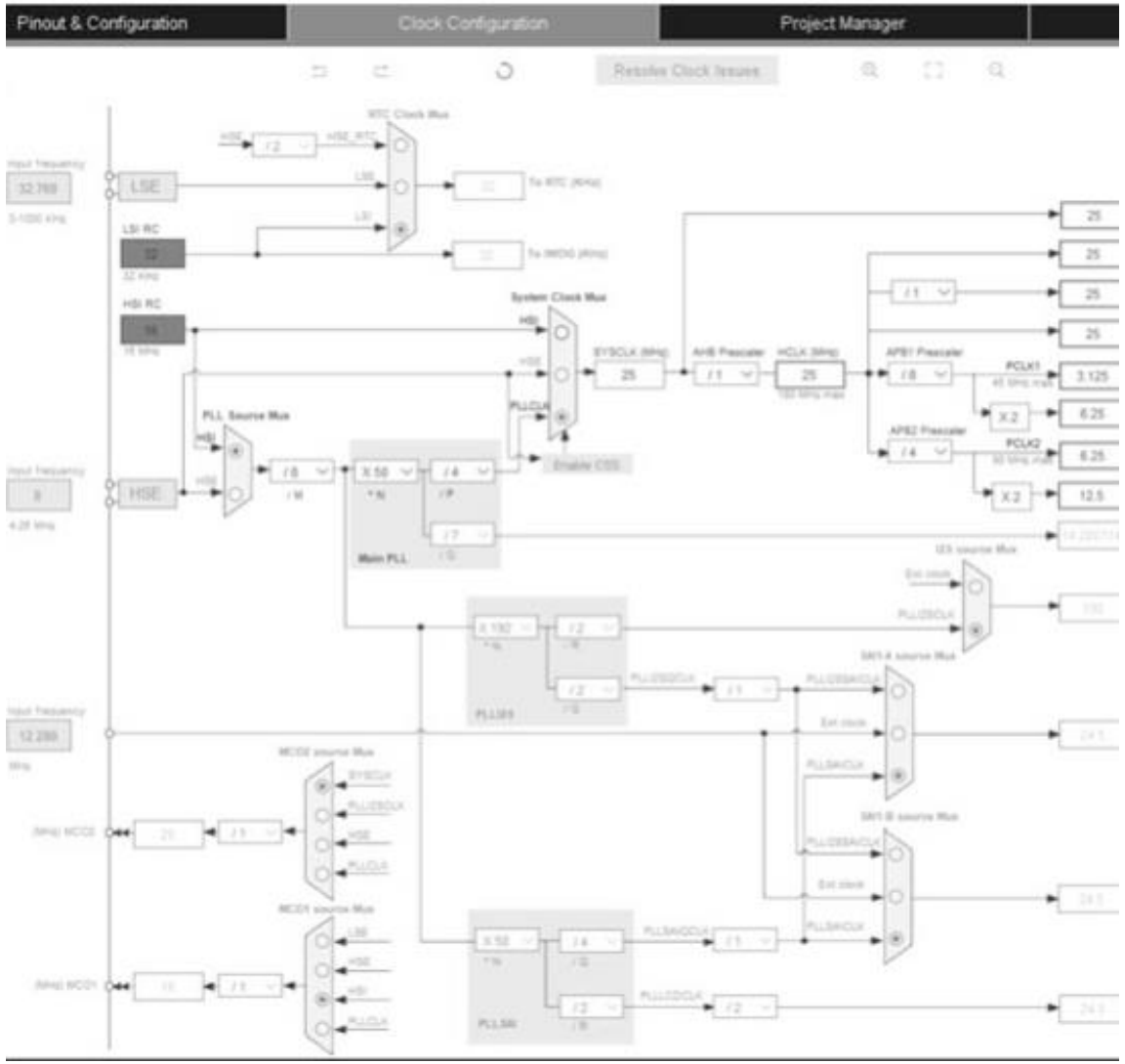
- A crystal oscillator circuit is given in Fig. Here, frequency of the clock signal is determined by the crystal which is a filter passing only signals with specific frequency.
- When the crystal oscillator is used, components other than the crystal and capacitors are placed within the microcontroller.
- However, the crystal cannot be placed inside the microcontroller. Therefore, it should be added to the system as an external unit.
- Crystal oscillators have relatively low response time. On the other hand, accuracy of crystal oscillators is superior compared to RC oscillators.

- In this figure, Q is the quartz crystal which is the main component of the oscillator.
- In simplest terms, the quartz crystal is represented by an RLC circuit with a parallel capacitor added to it.
- This setup leads to a frequency range that can be generated.
- Afterward, the exact clock signal frequency can be determined by the load capacitance value obtained by external capacitors *CL*1, *CL*2, and the stray capacitance *Cs* (not shown in the figure).
- The inverter circuit represented by the NOT gate in the figure aims to generate a phase shift. Hence, oscillation within the circuit can be generated.
- The feedback resistor R*F* , connected in parallel to the inverter, ensures the oscillator to work in linear region.



Managing Clocks in the STM32F4

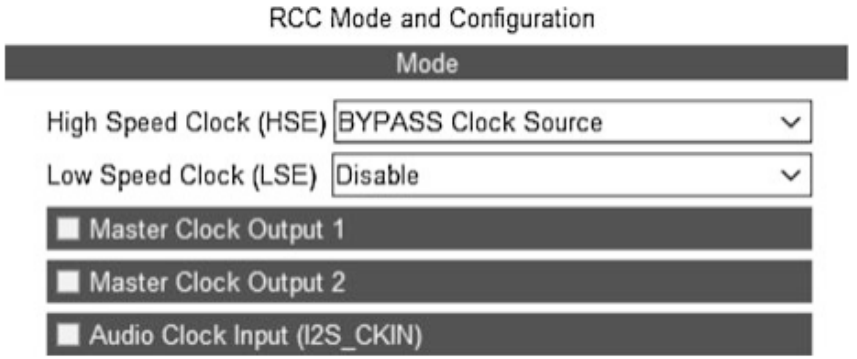
- The STM32F4 microcontroller has several clocks and clock signal sources. The reason for this is as follows. When more than one time-based operation is performed within the microcontroller, clocks working with different frequencies may be needed. In such operations, when one desired frequency is low, the other may be high. Therefore, different clocks should be used.
- Block diagram of the clock tree for the STM32F4 microcontroller, obtained from STM32CubeMX, is given in Fig. 6.4. As can be seen in this figure, all clocks used in the microcontroller can be derived from LSI, LSE, HSI, and HSE clocks.



- The STM32F4 microcontroller has two low speed clocks as LSI and LSE.
- LSI is the low speed internal clock generated from an internal 32 kHz RC oscillator.
- LSE is the low-speed external clock.
- There are two ways to generate it.
- The first one is connecting an external 32768 Hz crystal to the OSC32_IN and OSC32_OUT pins of the STM32 microcontroller.
- The second way is connecting an external clock to the OSC32_IN pin of the STM32F4 microcontroller.
- Here, the OSC32_OUT pin should be left at high impedance. In this setup, the clock frequency can be selected between 0 and 1000 kHz.
- Unfortunately, the STM32F4 board does not have an LSE crystal.

- The STM32F4 microcontroller has two high speed clocks as HSI and HSE.
- HSI is the high-speed internal clock generated from an internal 16 MHz RC oscillator.
- HSE is the high-speed external clock.
- There are two ways to generate it.
- The first one is connecting an external crystal to the OSC_IN and OSC_OUT pins of the STM32F4 microcontroller. This way, the clock frequency can be set between 4 and 26 MHz.
- The second way is connecting an external clock to the OSC_IN pin of the STM32F4 microcontroller. Here, the OSC_OUT pin should be left at high impedance. In this setup, the clock frequency can be set between 1 and 50 MHz.
- The STM32F4 board has an onboard 8 MHz HSE crystal.

- Low- and high-speed clocks introduced, can be selected from STM32CubeMX.
- To do so, the reader should select the RCC module from the “System Core” drop-down menu.
- The opened “RCC Mode and Configuration” window will be as in Fig.
- Here, HSE and LSE clock sources can be selected as “Disable,” “BYPASS clock source,” or “Crystal/Ceramic Resonator” from the related drop-down list.
- Also, two clock signals can be directed to output pins by selecting “Master Clock Output 1” and “Master Clock Output 2” boxes.
- Master clock output 1 signal can be fed by the LSE, HSE, HSI, or main PLL clock.
- Master clock output 2 signal can be fed by the SYSCLK, PLLI2S clock, HSE, or main PLL clock.
- Selected clock source frequencies can be divided by their prescalers.





- Other clocks used inside the STM32F4 microcontroller are as follows.
- The IWDG clock is used by independent watchdog timer and fed by the LSI clock.
- RTC clock is used by real-time clock module and fed by either the LSI, LSE, or scaled HSE clock.
- I2S clock is used by inter-IC sound peripheral unit and fed by either the PLLI2S or external I2S_CKIN pin. Here, PLLI2S can be fed by the HSI or HSE clock.
- SAI clocks are used by the synchronous audio interface peripheral unit and fed by either the PLLSAI, PLLI2S, or I2S_CKIN pin. Here, PLLSAI can be fed by the HSI or HSE clock. LCD-TFT clock is used by the LCD-TFT controller module and fed by PLLSAI.
- SYSCLK is the system clock. Hence, all clocks used inside the microcontroller, except the ones mentioned before, are derived from it. SYSCLK can be fed by the HSI, HSE, or main PLL. Its maximum frequency can be set to 180 MHz. Here, the main PLL can be fed by the HSI or HSE clock.
- HCLK is the processor clock used by CPU core, advanced high-performance bus (AHB), memory, and DMA. Its maximum frequency can be set to 180 MHz. HCLK is a gated clock which means it can be enabled or disabled by a control bit.
- FCLK is the free-running processor clock. This means it is not gated. In other words, it cannot be stopped by a control bit. FCLK continues to run in the background even if the CPU goes to sleep mode. Hence, it can control interrupts and events during this time.
- FCLK is also used for sampling interrupts and clocking debug blocks. It is synchronous to HCLK. PCLK1 is the low-speed advanced peripheral bus (APB1) clock. Its maximum frequency can be set to 45 MHz.
- APB1 timers use double PCLK1 frequency. PCLK2 is the high-speed advanced peripheral bus (APB2) clock. Its maximum frequency can be set to 90 MHz. APB2 timers use double PCLK2 frequency.

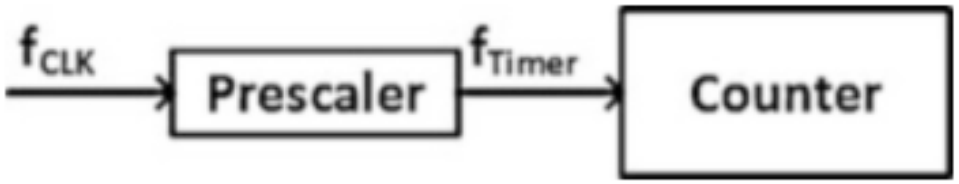
All prescalers used in the clock tree are actually frequency dividers. These are constructed using cascaded flip-flops. In operation, the input frequency is divided by two after each flip-flop. Hence, we can obtain f/2, f/4, f/8, · · · , frequency valued clock signals with this setup.

Although we are free to choose the mentioned clock sources, the best option is leaving them to be set by STM32CubeMX. Hence, we can benefit from the automatic clock configuration option there.

When one of the HCLK, FCLK, PCLK1, PCLK2, or system timer clock frequencies is entered to the related box in STM32CubeMX, all other clocks and PLL blocks are configured automatically if possible.

What Is a Timer?

- Timer is basically a counter such that the count value changes (going up, down, or up and down) by a trigger. If we pick the trigger to be the rising or falling edge of a periodic clock signal, then the count value changes based on the clock. This way, we can determine the actual time within our application running on the microcontroller.
- Assume that period of the clock signal fed to the timer module is $1/f_{Timer}$ s. We set the timer module to count up starting from zero.
- Assume further that the count reaches the value $(N - 1)$ at a specific time instant. Hence, we can deduce that the time passed between the start of the operation and specific time instant is N/f_{Timer} s.
- As can be seen in this example, we can deduce the actual time value by the help of the timer module. This allows performing time-based operations such as creating specific time intervals, periodically obtaining data, measuring the time duration between two events, or creating time related output by the microcontroller.



- We can also use the prescaler, basically a frequency divider, to change period of the clock signal fed to the timer module. This allows us to obtain different time values by the same clock signal. We provide a basic timer block composed of the prescaler and counter in Fig. In this figure, *fCLK* represents the frequency of the clock signal fed to the timer module. This signal is modified by the prescaler such that the new clock signal has frequency *fTimer*.

We next provide the actual usage example of the timer module. We know that the counter within the timer changes its value at every 1/*fTimer* s.

Let’s create a 10 s time interval using the clock signal with 180 MHz *fCLK* frequency.

We should count 1800 million clock signal edges to obtain the 10 s time interval unless we use the prescaler.

Instead, we can divide *fCLK* by 18000.

Hence, we can obtain the 10 s time interval by counting 100000 clock signal edges with frequency *fTimer*.

The timer module works independent of the CPU. Therefore, the CPU does not allocate its resources to determine the required time instant.

We can benefit from timer interrupts such that separately working CPU and timer module can achieve a time-based operation.

Let’s provide a simple example for this case. Assume that we want to toggle an LED every second.

The timer module can perform the counting process to reach 1 s in background without interfering the CPU.

As the toggling time is reached, the timer module generates an interrupt and the related callback function is executed by the CPU to toggle the LED.

The CPU can perform other tasks assigned to it except the callback function execution period.

Therefore, the CPU does not dissipate its valuable resources to the time calculation process.

Timers in the STM32F4

The STM32F4 microcontroller has four different timer types as base, watchdog, system timer (SysTick), and real-time clock (RTC).

These are used for specific purposes.

The base and system timer modules can be used for general time related operations.

The watchdog timer can be used for resetting the CPU when an error halts code execution.

The RTC can be used for determining the exact date and time. Hence, specific actions can be taken based on this information.

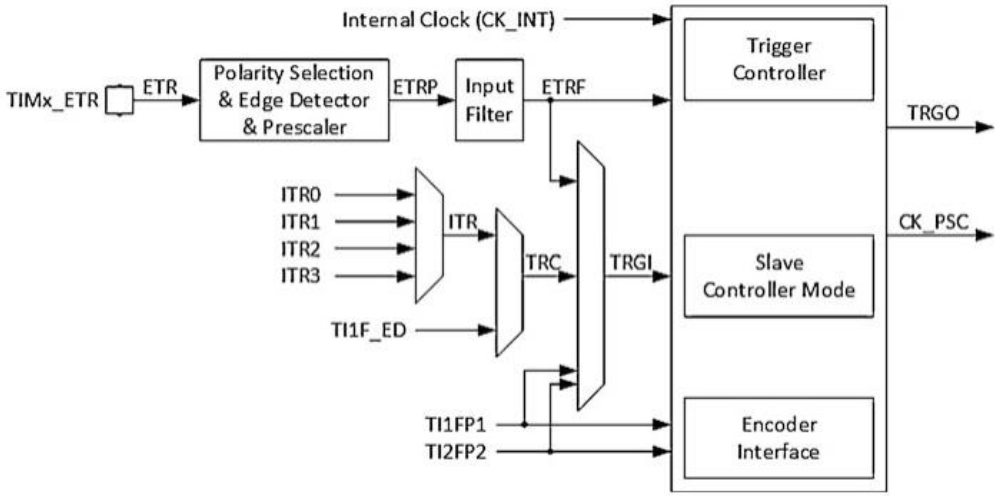


Fig. 6.7 Base timer trigger and clock controller block

- There are 14 base timers in the STM32F4 microcontroller. These are called TIM1, TIM2, · · · , TIM14. All of these timers share similar properties.
- The STM32F4 microcontroller base timer consists of four blocks as trigger and clock controller, counter, input capture, and output compare.
- The trigger and clock controller block in the base timer is used to start/stop, reset, or update the timer counter using one of the trigger inputs.

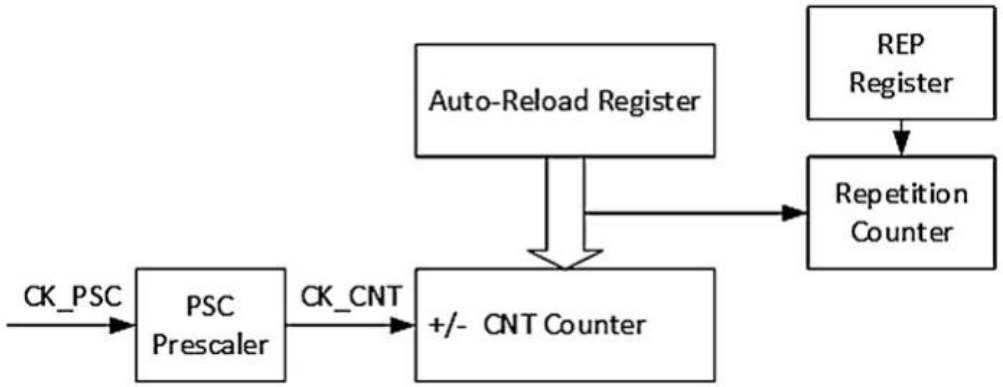


Fig. 6.8 Base timer counter block

- The counter block of the base timer is given in Fig. Here, the internal clock signal, CK_PSC, is divided by the 16-bit prescaler first to obtain the counter clock, CK_CNT. In every clock edge, content of the counter register, CNT counter, is updated according to the selected counting mode.
- The input capture block in the base timer is used to copy the value inside the counter register to the capture/compare register when the selected input signal transition is detected. This transition can be low to high (rising edge), high to low (falling edge), or both (rising and falling edges).
- The output compare block in the base timer is used for comparing the value inside the counter register with the capture/compare register. When both values are equal, transition of the output signal can be changed or an update event can be generated.

- As can be seen in Table 6.2, TIM6 and TIM7 have no capture/compare channels. Therefore, they are mainly used for triggering the DAC module or basic time operations with 16-bit counter. TIM1 and TIM8 are advanced control timers with complementary output. Hence, they can be used for generating three phase PWM signals with programmable inserted dead times. They also have all features of complete general-purpose timers. TIM2 and TIM5 have 32-bit counters. TIM3 and TIM4 have 16-bit counters. These four timers have four independent channels with full capture/compare modes. TIM9, TIM10, TIM11, TIM12, TIM13, and TIM14 have 16-bit counters. However, they can be only used in up mode. TIM9 and TIM12 have two capture/compare channels. The remaining timers have only one such channel.

Table 6.2 Summary of the STM32F4 microcontroller base timers

Timer	Resolution (bits)	Mode	Prescaler (bits)	Capture/compare channels	Max. timer clock (MHz)
TIM1, TIM8	16	Up, down, center aligned	16	4	180
TIM2, TIM5	32	Up, down, center aligned	16	4	90
TIM3, TIM4	16	Up, down, center aligned	16	4	90
TIM6, TIM7	16	Up	16	0	90
TIM9	16	Up	16	2	180
TIM10, TIM11	16	Up	16	1	180
TIM12	16	Up	16	2	90
TIM13, TIM14	16	Up	16	0	90

System Timer in the STM32F4 Microcontroller

- System timer, also called SysTick, is mainly used by the real-time operating system (RTOS). However, it can also be used as a dedicated timer. Since SysTick is extensively used by RTOS, it is fed by the main system clock. This clock can be divided by 1 or 8 according to the prescaler selection. SysTick has a 24-bit downcounter to create 1 ms time intervals.

Watchdog Timers in the STM32F4 Microcontroller

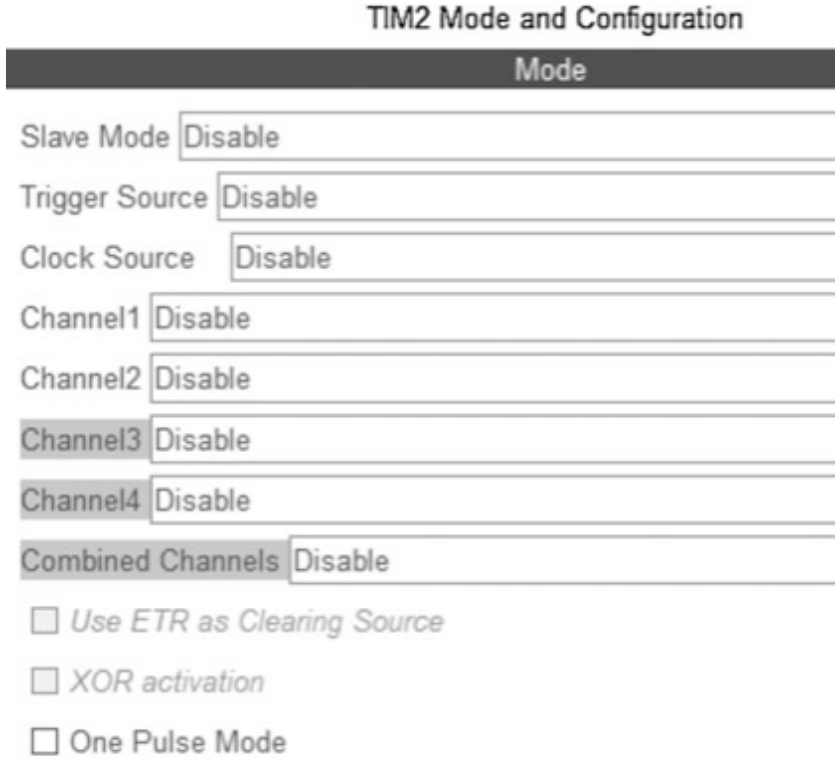
- A software failure can create an infinite loop and halt code execution within the microcontroller. Resetting the CPU is the best option to recover from such situations. Watchdog is a special timer which resets the CPU if its counter counts to zero from its initial value (called timeout). The counter must be reloaded by a code to prevent the reset operation. If the code cannot be executed until the reload operation, the watchdog timer resets the CPU.

Real-Time Clock in the STM32F4 Microcontroller

- Real-time clock (RTC) is a unique binary coded decimal (BCD) timer which can run independently as long as it is powered properly. It works even during reset operation or under low power modes. It keeps track of the exact time as seconds, minutes, hours (in 12- or 24-h format), day (day of the week), date (day of the month), month, and year in two 32-bit registers in BCD format. The subseconds is also tracked within the RTC and kept in another register. Daylight saving compensation can also be configured in RTC through software.

Timer Setup via C Language

- To set up base timers, go to the “Timers” drop-down menu in “Pinout and Configuration” window and select the desired TIMx peripheral under STM32CubeMX. Opening TIMx mode and configuration window will be as in Fig. 6.12. As mentioned before, different base timers may have different properties and capture/compare channels. Hence, this window may differ for the selected TIM peripheral.



TIMx mode and configuration window

- The system timer (SysTick) does not have a setup operation under STM32CubeMX. The main reason is that it runs in background independently. The user can modify only its related interrupt settings if it will be used.
- STM32CubeMX can be used to set up watchdog timers in the STM32F4 microcontroller. To do so, the user should go to the system core drop-down menu in the “Pinout and Configuration” window under STM32CubeMX.
- We can use STM32CubeMX to initialize the RTC. To do so, the reader should go to the Timers drop-down menu in the “Pinout & Configuration” window and select the RTC.

Timer Setup via C++ Language

- The base timer can be used under Mbed by its three classes Timer, Timeout, and Ticker. The timer class can be created by writing Timer TimerX. Here, TimerX is the name of the created Timer class and can be named as desired. The timer class is used to perform high-resolution counting operation by its 64-bit counter. It can be started, stopped, reset, or read for measuring precise time durations.
- The timeout class can be created by writing Timeout TimeoutX. Here, TimeoutX is the name of the created Timeout class and can be named as desired. The timeout class is used to generate an interrupt after a set delay time. This interrupt is generated only once.
- The ticker class can be created by writing Ticker TickerX. Here, TickerX is the name of the created Ticker class and can be named as desired. The ticker class is used to generate periodic interrupts at desired time intervals.

Mbed has a Watchdog class to control the IWDG watchdog timer. There is only one instance for the watchdog timer and can be used directly by Watchdog:: get_instance(). Or, this instance can be referenced as Watchdog &WatchdogX = Watchdog:: get_instance(). Here, WatchdogX is the name of the reference and can be named as desired. Then, this reference can be used throughout the code.

Mbed has no specific class to control RTC. Instead, RTC can be set to the desired date using the function set_time(time_t t). Here, t is the number of seconds since January 1, 1970, which is the universal UNIX timestamp. Instead of calculating this value to set the desired time, the function time_t mktime(tm *timeptr) can be used.

Timer Usage in C Language

- After setting the desired base timer properties in Sect. 6.3, related timer initialization code is created under the function MX_TIMx_Init. This function is called automatically in the “main.c” file.
- Then, specific HAL library functions can be used to control the base timer. The functions HAL_TIM_Base_Start and HAL_TIM_Base_Stop can be used to start or stop the base timer when it is not used with interrupts. This mode is used when capture/compare channels are used or base timer is used to trigger other peripherals.
- The functions HAL_TIM_Base_Start_IT and HAL_TIM_Base_Stop_IT can be used to start or stop the base timer when it is used with interrupts. This mode is used when specific time or time inter vals are created to perform the desired time-based tasks without polling CPU.
- HAL_TIM_PeriodElapsedCallback is the callback function which is executed when triggered by update event.

```
HAL_TIM_Base_Start(TIM_HandleTypeDef *htim)
/*
htim: pointer to the TIM_HandleTypeDef struct.
*/

HAL_TIM_Base_Stop(TIM_HandleTypeDef *htim)

HAL_TIM_Base_Start_IT(TIM_HandleTypeDef *htim)

HAL_TIM_Base_Stop_IT(TIM_HandleTypeDef *htim)

HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
```

- The functions HAL_TIM_IC_Start_IT and HAL_TIM_IC_Stop_IT can be used to start and stop input capture with interrupts on the selected channel. When the input capture is done, the callback function HAL_TIM_IC_CaptureCallback is executed. Inside it, captured values can be read using the function HAL_TIM_ReadCapturedValue.

```
HAL_TIM_IC_Start_IT(TIM_HandleTypeDef *htim, uint32_t Channel)
/*
htim: pointer to the TIM_HandleTypeDef struct.
Channel: variable used to select the channel for input capture
    operation. It can be entered as TIM_CHANNEL_x where x is the
    channel number.
*/

HAL_TIM_IC_Stop_IT(TIM_HandleTypeDef *htim, uint32_t Channel)

HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)

HAL_TIM_ReadCapturedValue(TIM_HandleTypeDef *htim, uint32_t
    Channel)
```

Usage of the System Timer

There is just one callback function named HAL_IncTick(void) for SysTick. This function is called every millisecond. We can use it to run time-based operations as such.

Usage of Watchdog Timers

After setting the desired watchdog timer properties in Sect. 6.3, related timer initialization code is created under the functions MX_IWDG_Init or MX_WWDG_Init based on the selected watchdog timer. The functions are called automatically in the “main.c” file. Afterward, specific HAL library functions can be used to control the watchdog timers. Functions HAL_IWDG_Refresh and HAL_WWDG_Refresh can be used to refresh IWDG and WWDG peripherals, respectively. More information on these functions are given next.

```
HAL_IWDG_Refresh(IWDG_HandleTypeDef *hiwdg)
/*
hiwdg: pointer to the IWDG_HandleTypeDef struct.
*/

HAL_WWDG_Refresh(WWDG_HandleTypeDef *hwwdg)
/*
hwwdg: pointer to the WWDG_HandleTypeDef struct.
*/
```

Usage of the RTC

After setting the desired RTC properties in Sect. 6.3, related timer initialization code is created under the function MX_RTC_Init. This function is called automatically in the “main.c” file. Then, specific HAL library functions can be used to control the RTC. Functions HAL_RTC_SetTime and HAL_RTC_SetDate can be used to set time of the day or exact calendar time. Functions HAL_RTC_GetTime and HAL_RTC_GetDate can be used to obtain the current day time or calendar time. More information on these functions are given next.

```
HAL_RTC_SetTime(RTC_HandleTypeDef *hrtc, RTC_TimeTypeDef *sTime,
    uint32_t Format)
/*
hrtc: pointer to the RTC_HandleTypeDef struct.
sTime: pointer to the Time struct.
Format: Time format is selected as binary with RTC_FORMAT_BIN or
    BCD with RTC_FORMAT_BCD.
*/

HAL_RTC_SetDate(RTC_HandleTypeDef *hrtc, RTC_DateTypeDef *sDate,
    uint32_t Format)

HAL_RTC_GetTime(RTC_HandleTypeDef *hrtc, RTC_TimeTypeDef *sTime,
    uint32_t Format)

HAL_RTC_GetDate(RTC_HandleTypeDef *hrtc, RTC_DateTypeDef *sDate,
    uint32_t Format)
```

Timer Usage Examples via C Language

- Our first example is toggling the red and green LEDs on the STM32F4 board. More precisely, the green LED, connected to pin PG13 of the board, and red LED, connected to pin PG14 of the board, will be toggled at 1 s intervals. There will be a 0.5 s delay between the toggling operations in the red and green LEDs.

Listing 6.1 Toggling the red and green LEDs using the TIM1 update and compare interrupts, in C language

```
/* USER CODE BEGIN 0 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
}

void HAL_TIM_OC_DelayElapsedCallback(TIM_HandleTypeDef *htim)
{
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
if (__HAL_TIM_GET_FLAG(&htim1, TIM_FLAG_UPDATE) != RESET)
__HAL_TIM_CLEAR_FLAG(&htim1, TIM_FLAG_UPDATE);
HAL_TIM_Base_Start_IT(&htim1);
HAL_TIM_OC_Start_IT(&htim1, TIM_CHANNEL_1);
/* USER CODE END 2 */
```

- To do so, we use the external HSE clock. Then, the SYSCLK frequency is set to maximum 180 MHz in the clock configuration window. Afterward, we enable the base timer TIM1 by selecting the internal clock as the clock source. We set the prescaler to 3599, counter period to 49999, and counter mode to up mode to obtain ((49999+1) × (3599+1))/180000000 = 1 s update event period. The green LED is toggled using this update event. Also, Channel 1 is configured as “Output Compare No Output.”
- We picked this mode to use the output compare mode to create the desired delay between toggling red and green LEDs. Here, we do not create an output signal. After the “Output Compare No Output” is selected from the related drop-down list, related parameters are added to the “Parameter Settings” window. Among these, “Pulse” value should be set to 25000 to obtain (25000 × (3599+1)) / 180000000 = 0.5 s delay time. Finally, we enable the TIM1 update interrupt and TIM1 capture/compare interrupt from the “NVIC Settings” window.

Listing 6.1 Toggling the red and green LEDs using the TIM1 update and compare interrupts, in C language

```
/* USER CODE BEGIN 0 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
}

void HAL_TIM_OC_DelayElapsedCallback(TIM_HandleTypeDef *htim)
{
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
if (__HAL_TIM_GET_FLAG(&htim1, TIM_FLAG_UPDATE) != RESET)
__HAL_TIM_CLEAR_FLAG(&htim1, TIM_FLAG_UPDATE);
HAL_TIM_Base_Start_IT(&htim1);
HAL_TIM_OC_Start_IT(&htim1, TIM_CHANNEL_1);
/* USER CODE END 2 */
```

- We provide the C code for our first example in Listing 6.1. Please add the given code parts to the related locations in your “main.c” file.
- Here, we use the code line if(__HAL_TIM_GET_FLAG(&htim1, TIM_FLAG_UPDATE) != RESET) __HAL_TIM_CLEAR_FLAG(&htim1, TIM_FLAG_UPDATE) to clear the update event flag in case it is set before the timer has started.
- When the timer init function is executed, this flag is set beforehand and must be cleared to prevent erroneous run.

- When the code in Listing 6.1 is executed, the red LED toggles first after 0.5 s using the compare interrupt. Then, the green LED toggles after 1 s using the update interrupt. Then, they keep toggling with 1 s intervals and 0.5 s between them.
- Our second example is on turning on and off the green LED, connected to pin PG13 of the STM32F4 board. More precisely, the green LED will be turned on if pressing time of the user button, connected to pin PA0 of the STM32F4 board, is more than 3 s. If this time is less than 3 s, then the green LED turns off.
- To do so, we use the internal HSI oscillator to generate the 180 MHz SYSCLK. Afterward, we enable the base timer TIM2 by selecting Channel 1 in the “Input Capture” direct mode. We use Channel 1 of TIM2 since the user button on the STM32F4 board is connected to this channel. We select “Polarity Selection” as “Both Edges” from “Input Capture Channel 1” drop-down list.
- We also set the “Prescaler” to 0 and “Counter” period to 0xFFFFFFFF to obtain (0xFFFFFFFF + 1)/(PCLK1 × 2) = 47.72 s capture time. Here, we use PCLK1 × 2 since TIM2 is fed by twice the PCLK1 frequency. Finally, we enable the capture/compare interrupt by selecting TIM2 global interrupt box in the “NVIC Settings” window.

Listing 6.2 Measuring the button pressing time using the input capture mode, in C language

```
/* USER CODE BEGIN PV */
uint32_t icRisingEdgeValue = 0;
uint32_t icFallingEdgeValue = 0;
uint32_t icDifferenceValue = 0;
float pressingTime = 0;
uint8_t icFlag = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
if (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
{
if (icFlag == 0)
{
icRisingEdgeValue = HAL_TIM_ReadCapturedValue(&htim2,
    TIM_CHANNEL_1);
icFlag = 1;
}
else
{
icFallingEdgeValue = HAL_TIM_ReadCapturedValue(&htim2,
    TIM_CHANNEL_1);
if (icFallingEdgeValue > icRisingEdgeValue)
{
icDifferenceValue = icFallingEdgeValue - icRisingEdgeValue;
}

else
{
icDifferenceValue = 0xFFFFFFFF + 1 + icFallingEdgeValue -
    icRisingEdgeValue;
}
pressingTime = ((float)icDifferenceValue) / (2 *
    HAL_RCC_GetPCLK1Freq());
if (pressingTime > 3.0)
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
else
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
icFlag = 0;
}
}
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
HAL_TIM_IC_Start_IT(&htim2, TIM_CHANNEL_1);
/* USER CODE END 2 */
```

- We provide the C code for our second example in Listing 6.2. Please add the given code parts to the related places in your “main.c” file. When the user button on the STM32F4 board is pressed, its output goes from low to high and captured value is written to the variable icRisingEdgeValue. When the user button is released, its output goes from high to low and captured value is written to the variable icFallingEdgeValue. Then, we calculate the difference between these two values. If the second value is smaller than the first one, it means that counter is reset after first capture. Hence, the constant 0xFFFFFFFF + 1 is added to the difference. As the time duration of the user button press is calculated this way, the green LED is turned on or off accordingly.

- In our third example, the green LED connected to pin PG13 of the STM32F4 board is toggled at 1 s intervals using SysTick. To do so, we use the external HSE clock. Then, the SYSCLK frequency is set to 180 MHz and “Cortex System Timer” is set to the same value by setting its prescaler to 1. We provide the C code for our third example in Listing 6.3. Please add the given code parts to the related places in your “main.c” file.

Listing 6.3 Toggling the green LED using SysTick, in C language

```
/* USER CODE BEGIN PV */
uint16_t sysTickCnt = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_IncTick(void)
{
sysTickCnt++;
if (sysTickCnt >= 1000)
{
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
sysTickCnt = 0;
}
}
/* USER CODE END 0 */
```

Listing 6.6 Toggling the green and red LEDs using timer interrupts, in C++

```
#include "mbed.h"

DigitalOut greenLED(LED1);
DigitalOut redLED(LED2);
Timeout timeoutDelay;
Ticker ticker1;
Ticker ticker2;

void toggleGreenLED()
{
greenLED = !greenLED;
}

void toggleRedLED()
{
redLED = !redLED;
}

void delay05()
{
toggleRedLED();
ticker1.attach(&toggleRedLED, 1s);
}

int main()
{
timeoutDelay.attach(&delay05, 500ms);
ticker2.attach(&toggleGreenLED, 1s);

while (1);
}
```

- Mbed does not have a function to be used for the output compare mode for timing purposes. Hence, we used two ticker classes to toggle the red and green LEDs in 1 s intervals in our example. We also used another timeout class to create 0.5 s delay between the red and green LEDs.
- In Listing 6.6, timeoutDelay is the created Timeout class, and it calls the function delay05() once to create 0.5 s delay between LEDs. In the function delay05(), the red LED connected to pin PG14 of the board is toggled using the function toggleRedLED(). ticker1, the created Ticker class, is also attached to the function toggleRedLED() to toggle the red LED with 1 s intervals. ticker2, the second created Ticker class, is attached to the function toggleGreenLED() at the beginning of the main function to toggle the green LED with 1 s interval.

- We next repeat the second example given in Sect. 6.4.1 now in C++ language. Mbed does not have a function for the input capture mode. Hence, we use the GPIO interrupts and Timer class to measure the duration between successive interrupt requests. We provide the corresponding C++ code for this example in Listing 6.7.
- In Listing 6.7, the timer1 is started at the rising edge of the GPIO interrupt generated by the user button press. At the falling edge of the user button press, the timer1 is stopped and elapsed time is measured in milliseconds. If the elapsed time is greater than 3000 ms, then the green LED on the STM32F4 board turns on. Otherwise, the green LED turns off. Finally, the timer1 is reset to measure the next pressing time.

Listing 6.7 Measuring button pressing time using GPIO interrupts and timer, in C++ language

```
#include "mbed.h"

InterruptIn userButton(BUTTON1);
DigitalOut greenLED(LED1);
Timer timer1;

uint32_t elapsedMs;

void triggerRise()
{
timer1.start();
}

void triggerFall()
{
timer1.stop();
elapsedMs = chrono::duration_cast<chrono::milliseconds>(timer1.
    elapsed_time()).count();
if (elapsedMs > 3000)
greenLED = 1;
else
greenLED = 0;
timer1.reset();
}

int main()
{
userButton.rise(&triggerRise);
userButton.fall(&triggerFall);

while (1);
}
```

Analog and Digital Values

- In order to explain analog values better, let's take temperature readings of a room. We can measure the temperature value at any time instant. Hence, we can have infinite number of observations at hand. Each observation can be represented by infinite precision depending on the measuring device. In short, we can obtain infinite number of observations, each having infinite precision. These are the two defining properties of analog values.
- Temperature is not the only analog value in real world. In fact, measurements taken from actual physical systems will be in analog form most of the times. We will need a sensor to make such measurements. Although sensors differ based on their application areas, they share a common property. A sensor converts one value to another form. For our purposes, we will pick sensors converting a physical measurement to electrical form either as voltage or current. Hence, it can be processed in the embedded system. We will introduce specific sensors for the application at hand whenever needed throughout the book.

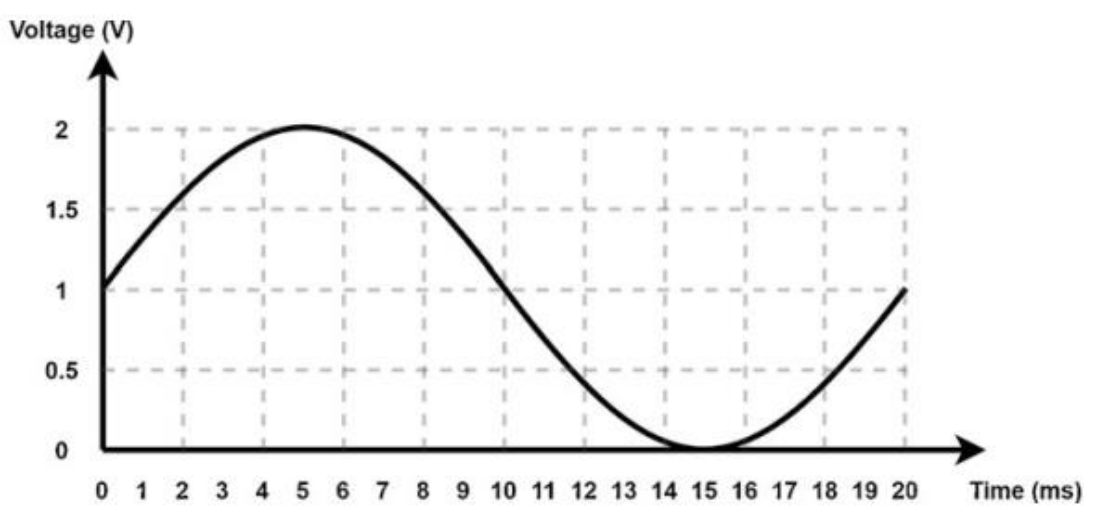


Fig. 7.1 A sample analog signal

- We can represent analog values in time. Such a representation will be called analog signal. We provide a sample analog signal in Fig. 7.1. The reader should always remember that this signal is composed of infinite number of values. Amplitude of the signal at any time instant will have infinite precision as well.

Digital Values in Embedded Systems

- An embedded system has limited memory space and processing power by its nature. In other words, we cannot store or process infinite numbered and ranged analog values in the embedded system. The suitable form for such systems is digital. The basic digital representation is the bit which can take two values as logic level 0 or 1.
- Instead of using the analog voltage there, its level is used. Therefore, all processes based on the bit are based on predefined voltage levels, not the exact analog voltage value. This is the main difference of digital representation.
- As in analog signals, we can have digital signals. Here, we assume that we are keeping successive digital values in successive memory locations. We provide a sample digital signal in Fig. 7.2.

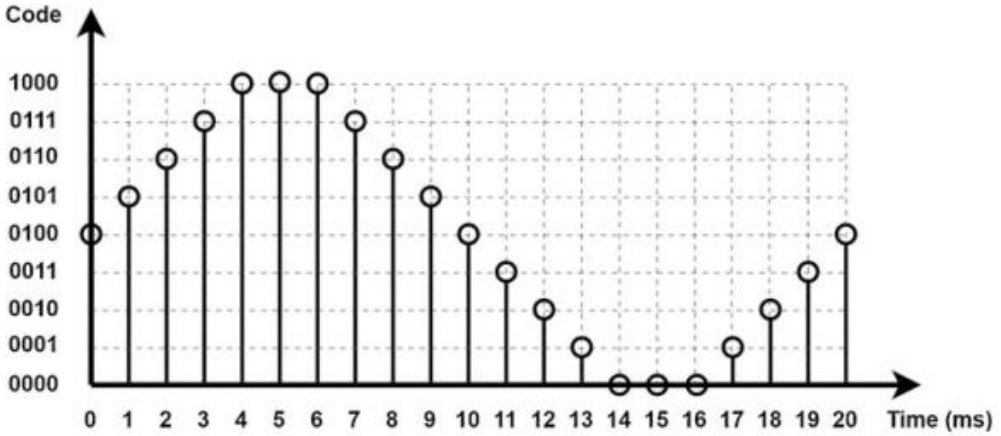


Fig. 7.2 A sample digital signal

Analog to Digital Conversion in Embedded Systems

In order to process an analog value in an embedded system, we should convert it to digital form. Hence, the digital value will have limited range. Besides, if we intend to obtain a digital signal from its analog counterpart, then we will have limited samples as well. The ADC module within the embedded system is responsible for this operation.

The ADC module performs two operations as sampling and quantization. We will explain these in detail in this section. Then, we will focus on the STM32F4 microcontroller as the embedded system and explain all the steps within it related to the ADC operation

Sampling

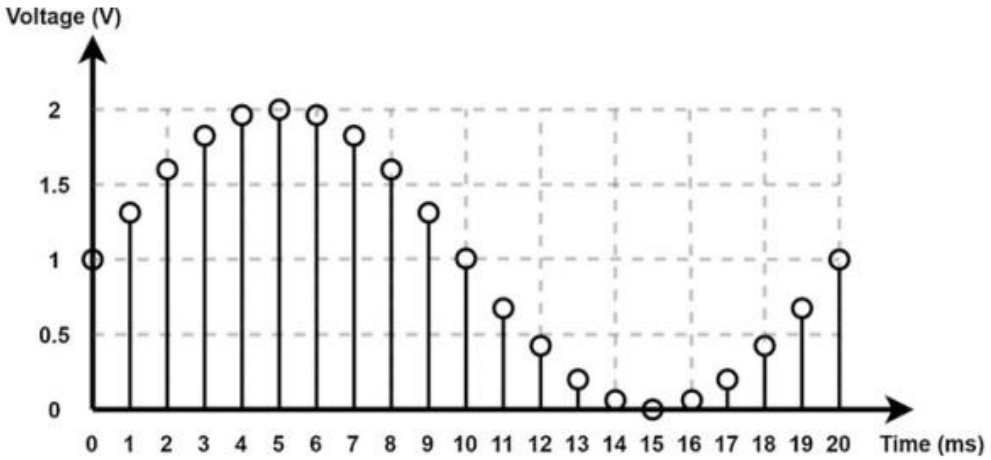


Fig. 7.3 Sampled signal

- Sampling is the first step in analog to digital conversion. The analog value is acquired in time by sampling. In other words, the analog value is observed at a certain time instant.
- The sampling operation is performed in a periodic manner most of the times. Therefore, we acquire successive samples of the analog signal in time. We will need a clock signal for this operation.
- The most important issue here is the sampling rate in which how many samples are acquired in 1 s. This is called sampling frequency represented by Hz. The duration between two samples is called the sampling period (one over the sampling frequency).
- Let's provide a simple example on the sampling operation. Assume that the analog voltage at an electrical circuit is as in Fig. 7.1. This is in fact an analog signal with 2 V peak to peak and 1 V DC offset value. We would like to take samples of this signal at every millisecond. Hence, we will have observations of the signal by the 1 kHz sampling frequency. Then, the sampled signal becomes as in Fig. 7.3.

Quantization

- As we sample the signal, we obtain its values at certain time instants. However, the value of each sample still has infinite range. Since we cannot store such a value in the embedded system, we should map it to finite range. Quantization performs this operation. Let's see how this can be done.
- First and most important of all, there should be minimum and maximum voltage values as limits in the quantization operation. This is called the full-scale range (FSR).
- Second, we should allocate certain number of bits to represent the analog value. The number of allocated bits indicates the total number of levels we can use to represent the analog value. To be exact, *N* number of bits will result in a total of 2*N* levels. Each level in quantized form corresponds to a bin having certain width in analog form. Therefore, all analog values within the bin are represented by the same digital value.

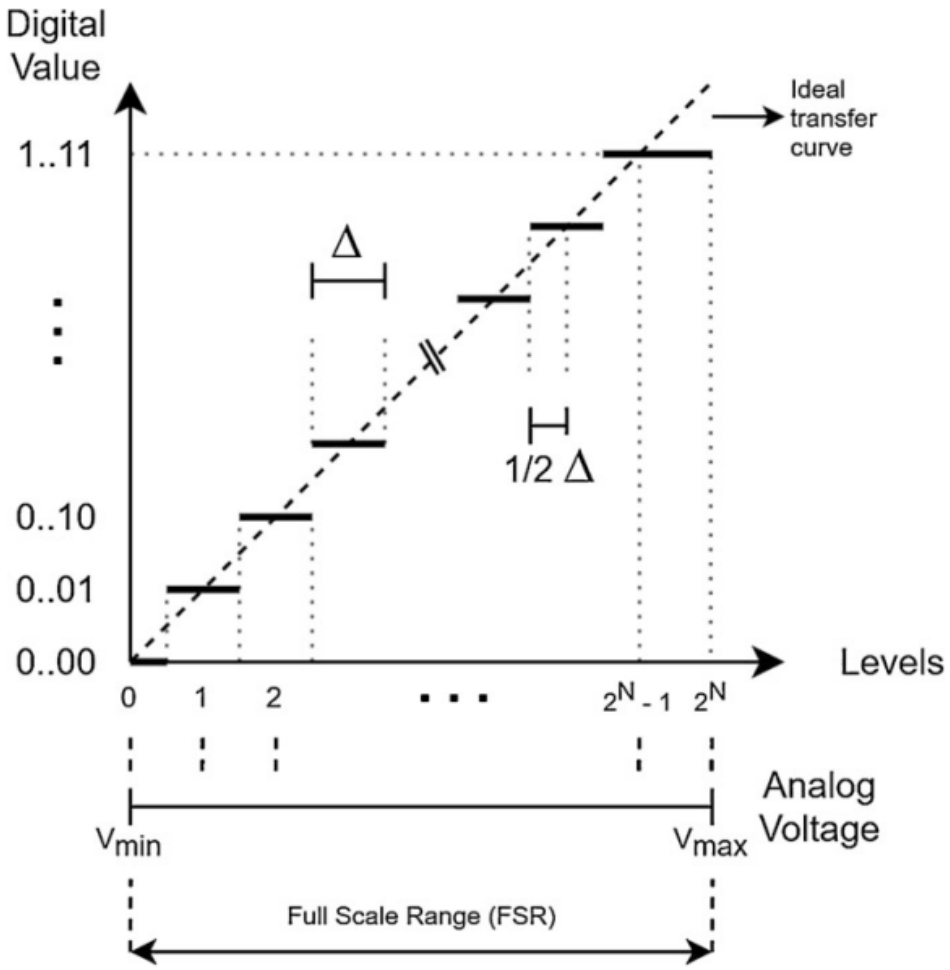


Fig. 7.4 The quantization operation

- This will be the case for all bins in the quantization operation. We can represent the bin width based on FSR and number of levels as = *FSR/2N*. The bin width is also directly related to the resolution such that a higher quantization level corresponds to better resolution with the same FSR value. We schematically show all these definitions in Fig. 7.4.

We can represent the quantization operation for the STM32F4 microcontroller as follows. The FSR for this microcontroller is set by the positive and negative reference voltage values V_{ref+} and V_{ref-}. The negative reference voltage is set to VSSA (ground voltage) for most general-purpose microcontrollers. This is also the case for the STM32F4 microcontroller. Hence, we will have $FSR = V_{ref+}$. The maximum number of bits assigned to the result of the ADC operation is 12 in the STM32F4 microcontroller. Hence, $N = 12$ here. This leads to

$$\Delta = \frac{V_{ref+}}{2^{12}} \tag{7.1}$$

The ADC operation represents the lowest analog voltage bin range, Δ, with the digital value zero. Likewise, the highest analog voltage bin range, $V_{ref+} - \Delta$, will have the corresponding digital value as 2^{N-1}. Based on these limits, we can obtain the digital (quantized) value for a given input voltage V_{in} as

$$Q = \left\lfloor \frac{V_{in}}{\Delta} \right\rfloor \tag{7.2}$$

where $\lfloor \cdot \rfloor$ indicates the floor function. We should note that when $V_{in} = V_{ref+}$ we will still have $Q = 2^{N-1}$. Although this case could have been formalized in Eq. 7.2, it would complicate the formula. Therefore, we handled it as a separate case.

Since we assign 12 bits for quantization, the quantized value can be represented in unsigned integer with 16 bits, `unsigned short` or `uint16_t` format, as mentioned in Sect. 7.1.3. Since 12 is less than 16, the most significant 4 bits will be set to zero. As a result, we have an unsigned number to represent the input analog voltage.

After obtaining the digital (quantized) value of an input voltage, we can convert it to `float` form by

$$Q_f = \frac{Q}{2^N} V_{ref+} \tag{7.3}$$

Please remember that even in this case, we will have quantized levels. However, each quantization level will be in `float` form.

Let's provide three examples on the quantization operation. In our examples, we will take $V_{ref+} = 3.3$ V and $N = 12$. First, assume that $V_{in} = \Delta/2$. The digital value corresponding to this input voltage will be 0. Hence, it can be represented in `unsigned short` form as `0x0000`. Second, assume that $V_{in} = 3.3/2$. The digital value corresponding to this input voltage will be 2048. Hence, it can be represented in `unsigned short` form as `0x0800`. Third, assume that $V_{in} = 3.3 - \Delta/4$. The digital value corresponding to this input voltage will be 4095. Hence, it can be represented in `unsigned short` form as `0x0FFF`.

ADC Operation in the STM32F4 Microcontroller

- Up to now, we explained sampling and quantization as if they are two separate operations. In practice, they are handled together by the ADC module. Besides, ADC modules in different embedded systems have their limitations. Therefore, we pick the STM32F4 microcontroller and explain working principles of its ADC module next.
- There are three ADC modules in the STM32F4 microcontroller called ADC1, ADC2, and ADC3. These modules can work independently. Therefore, we can configure the sampling frequency used in the ADC operation for each module separately. These modules provide 12-bit output. Hence, they are called 12-bit ADC modules. Based on the definitions in the previous section, we know that the total number of levels these ADC modules provide is 212. Finally, each ADC module works by the successive approximation register (SAR) methodology in analog to digital conversion.

- General block diagram of one ADC module in the STM32F4 microcontroller is as in Fig. 7.5. We will next explore the steps in this block diagram in detail. Therefore, we will cover input sources, triggers, modes, and sampling and clocks next. To note here, we will only cover one ADC module from this point on since the other two have exactly the same properties.

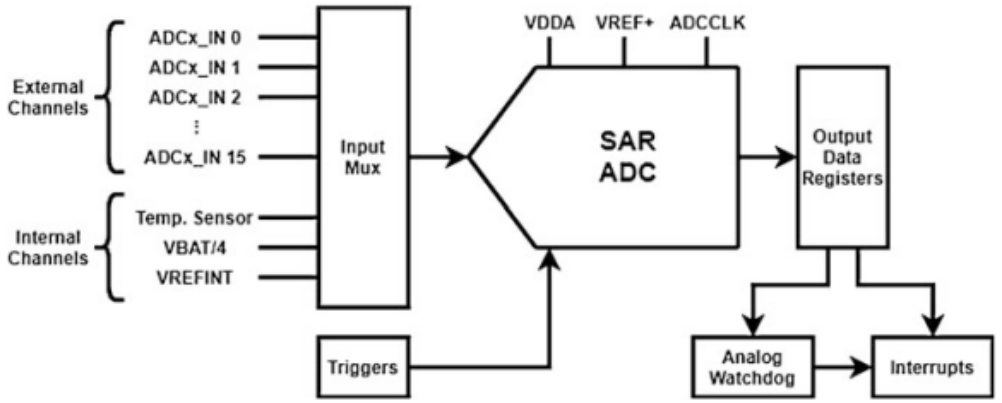


Fig. 7.5 Block diagram of the ADC module in the STM32F4 microcontroller

Input Sources

- The ADC module in the STM32F4 microcontroller has up to 19 input channels (input sources) as being internal and external. External channels are the GPIO pins that can be used for connecting external sources. We tabulated GPIO pins of the STM32F4 microcontroller, available on the STM32F4 board, that can be used for ADC input in Sect. 2.3.2. Internal channels are connected to the internal reference voltage (Vrefint), internal temperature sensor (Vsense), and Vbat monitoring (Vbat/4) pins of the microcontroller.
- The STM32F4 microcontroller ADC module also has voltage supply inputs as VDDA and VSSA. They are used to feed power to the module. The VSSA pin of the ADC module is connected to ground externally on the STM32F4 board. The VDDA pin of the ADC module is connected to VDD pin of the STM32F4 board externally with a noise filter. Hence, other digital operations do not add noise during the conversion operation.
- Besides, the ADC module has Vref+ and Vref- inputs. These define the FSR in the analog to digital conversion operation. The Vref- pin is connected to ground internally in the STM32F4 microcontroller. The Vref+ pin is connected to VDDA which is also connected to VDD in the STM32F4 board. Hence, the FSR for the ADC operation becomes VDD.



Triggers

- The ADC operation can start in two different ways. First, the ADC module can be triggered by software via code snippet. Second, the ADC module can be triggered by hardware using a peripheral unit such as GPIO or timer. As for the software trigger, the ADC module should be set up first. Then, we can use a special register bit which starts the conversion. This bit can be accessed by the HAL library or Mbed functions in the user code.
- If we want to use the hardware trigger, then we should wait for an event (interrupt) to occur. The hardware trigger edge polarity can be selected as rising, falling, or both rising and falling. This means that the ADC module will be triggered when the trigger signal goes from low to high, from high to low, or in both transitions. We can use the EXTI_11 line, connected to 11th pin of the GPIO ports, for generating a hardware trigger. Also, the timer capture/compare events or timer trigger out events (interrupts) can be used for this purpose.

Operating Modes

- The ADC module can work in different operating modes as single-shot, continuous, or discontinuous. These modes have their configurations as single or scan (multi) channel. We provide them as a flowchart in Fig. 7.6

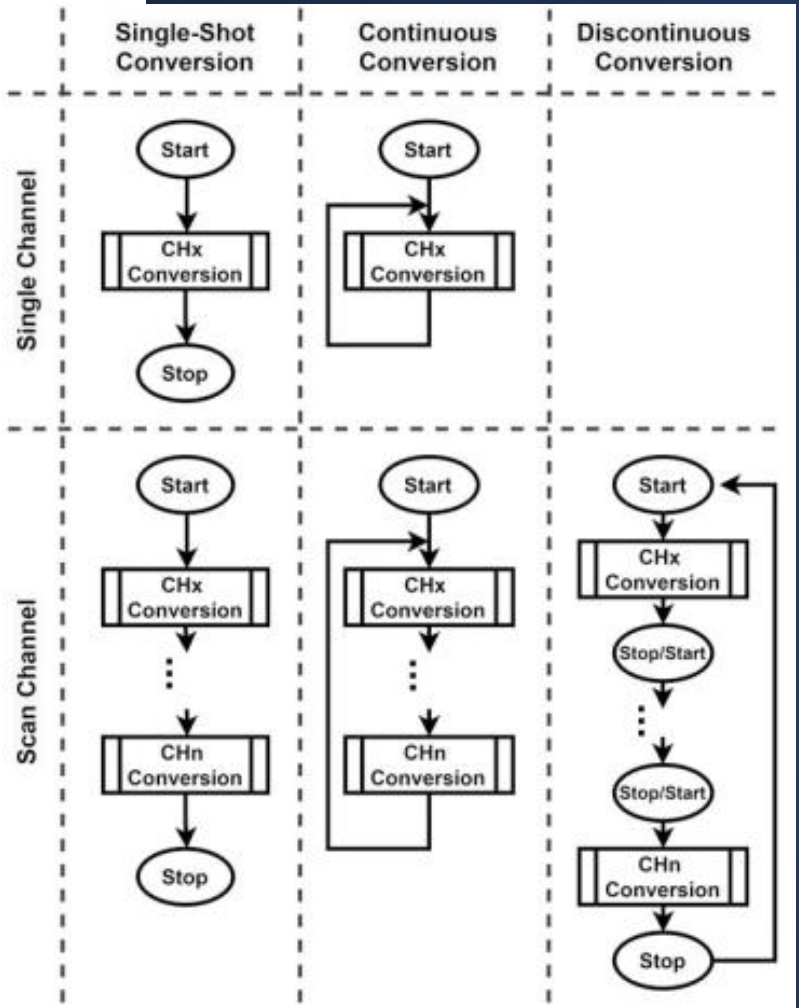


Fig. 7.6 Flowchart of the ADC module operating modes

- The ADC module performs conversion using a single input in the single channel mode. In the scan (multi) channel mode, there can be more than one input to the ADC module. The module performs conversion using these sequentially (one after the other).
- The ADC module stops after performing one conversion in the single-shot mode. If the scan channel mode is selected in this mode, then the conversion is performed for each channel. Afterward, the conversion stops. If a new conversion is required in the single-shot mode, then the module should be triggered again. In the continuous mode, the ADC module performs conversion automatically without re-triggering until the user stops it within the code. There is also discontinuous mode which can be only used if channel conversion mode is configured as scan channel. In this mode, the conversion stops after each channel while performing the conversion using multiple channels. The ADC module should be triggered to move on to the next channel.

ADC Interrupts

- The ADC module can generate an interrupt either at the end of conversion or by its analog watchdog property. These interrupts should be enabled separately by their respective bits.
- The end of conversion interrupt is generated separately for the regular or injected modes. Their corresponding callback functions are also separate.
- The ADC result can be compared with two threshold values set as low and high via the analog watchdog feature. This is done automatically after setting up the ADC module.
- Hence, there is no need to run a separate code for the operation. If the conversion is outside the threshold values, then an interrupt can be generated. This operation is a sort of software configurable window comparator. The advantage of this operation is that it does not need an analog reference voltage. The comparison is done by the set threshold values.

Sampling and Clocks

- The ADC module has its dedicated clock, ADCCLK, for this purpose. The maximum clock speed for ADCCLK is 36 MHz.
- The total conversion time for the ADC module is the sum of configured sampling period plus SAR operation duration depending on data resolution. The sampling period can be one of 3, 15, 28, 56, 84, 112, 144, or 480 ADC clock cycles. The SAR operation duration is 15 ADC clock cycles for 12-bit data resolution. This duration can be reduced to 13, 11, or 9 ADC clock cycles for 10-, 8-, and 6-bit data resolutions, respectively. As a result, the ADC module can produce 2.4 mega samples per second with 12-bit resolution. It is possible to go up to 4 mega samples per second with lower (6-bit) resolution by setting the ADCCLK to work at 36 MHz.

Obtaining the ADC Operation Result

- As the ADC operation ends, the conversion result is written to the ADC_DR or ADC_JDRx register based on the regular or injected operation, respectively. These registers keep 16-bit data. Since the conversion result can be 12 to 6 bits, it should be aligned to right or left by a dedicated register.
- The alignment works differently for the regular and injected ADC operations. For the right aligned result, zeros or sign bits are added as prefix for the regular or injected operations, respectively. For the left aligned result, zeros are added as suffix for the regular or injected operations. There is one exception here. If a 6-bit conversion is performed, the first byte is not used in operation. The second byte is filled with zeros as suffix for the regular conversion. This is also the case for injected mode, in which the most significant bit in the second byte is kept for the sign of the operation. As a side note, the injected mode can be used to obtain negative results.

ADC Setup via C Language

- We will benefit from STM32CubeMX to set up the ADC module properties in C language. In order to set up ADC via the STM32F4 microcontroller, first open the “Device Configuration Tool” window by clicking the “.ioc” file in the project explorer window. Then, click on one of ADC1, ADC2, or ADC3 under the “Analog” section from the left menu of STM32CubeMX.
- If the internal temperature sensor, Vbat or Vrefint channel, will be selected as the ADC input source, then enable the channel under the “Analog - ADCx” section as in Fig. 7.9. Here, x can be 1, 2, or 3 to denote ADC1, ADC2, and ADC3, respectively.



g. 7.9 ADC mode menu in STM32CubeMX

- If an external source will be connected to the ADC module, then click on the GPIO pin with ADC input capability on the STM32F4 microcontroller in the STM32CubeMX interface. Afterward, select the GPIO type as “ADCx_INy” in the “Device Configuration Tool” window as in Fig. 7.10. To note here, y can be any value between 0 and 15. Finally, enable the channel under the “Analog - ADCx” section.

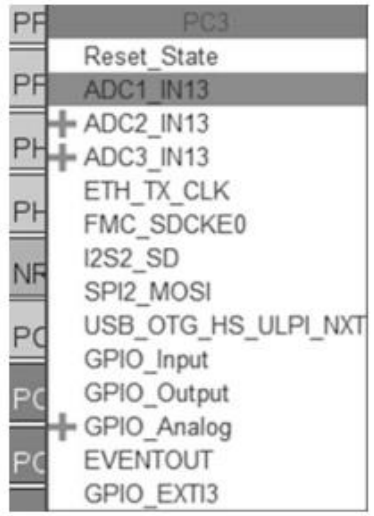


Fig. 7.10 ADC input source configuration in STM32CubeMX

Setting Up Operating Modes

- Once the input source is selected, ADCx settings can be configured from the opened “Parameter Settings” tab in the “Configuration” menu as in Fig. 7.11. By default, the ADC module is configured to work in single-shot conversion mode triggered by software in 12-bit right aligned data mode. Data width can be selected as 12, 10, 8, or 6 bits. Data alignment can be selected as left or right aligned from the configuration menu.

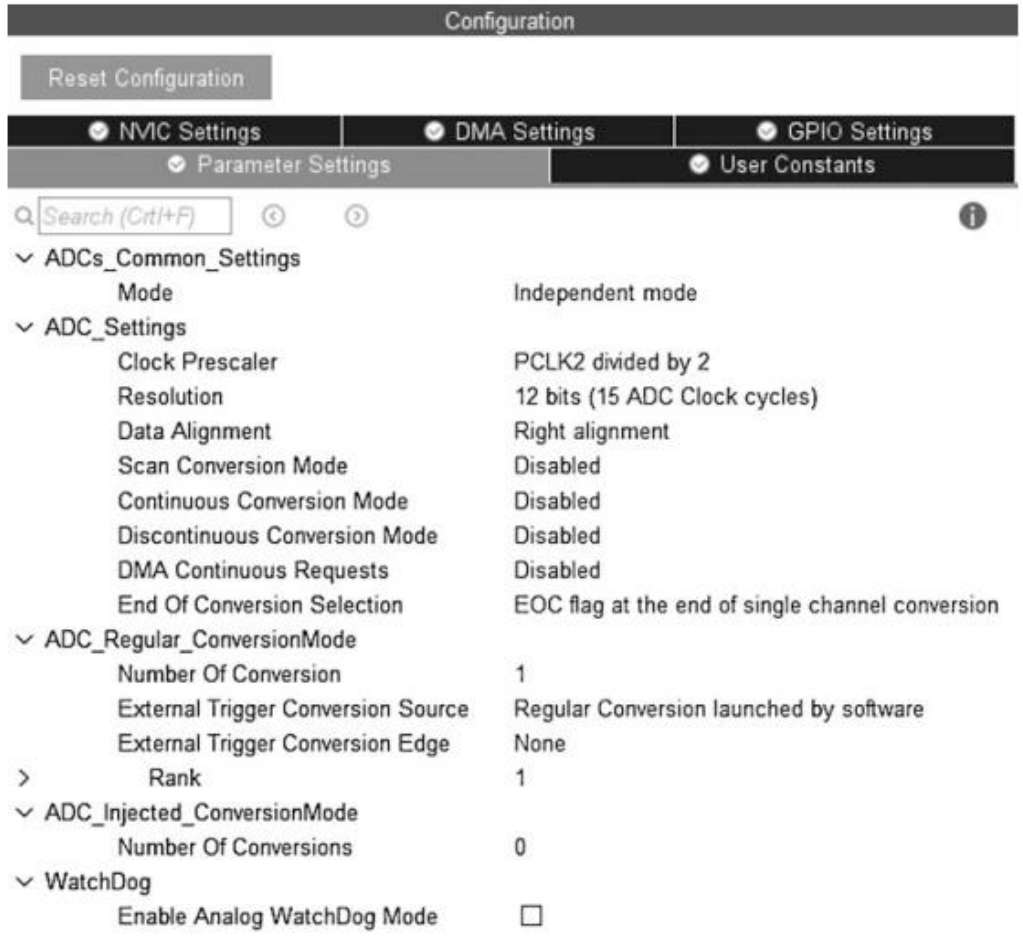


Fig. 7.11 ADC configuration menu in STM32CubeMX

We can enable the scan mode if more than one input channel is used in analog to digital conversion. To do so, we should first select more than one input channel as ADC input. The selected and configured pins should all belong to the same ADC module such as ADC1_IN1 or ADC1_IN2. Then, the “Number of Conversion” option under the “ADC_Regular_ConversionMode” setting should be set to the number of selected input channels in the configuration menu as in Fig. 7.11.

Afterward, there will be multiple “Rank” options in the configuration menu and the “Scan Conversion Mode” will be enabled automatically. In each rank option, the channel number should be selected in the order that the conversion will occur. The sampling time can be changed under the “Rank” options individually for each channel.

The ADC will perform a single-shot conversion by default. If we would like to perform continuous or discontinuous conversion, we should enable the “Continuous Conversion Mode” or “Discontinuous Conversion Mode” in the configuration menu as in Fig. 7.11. To note here, the discontinuous conversion mode can only be enabled if the scan channel mode has been selected

Setting Up ADC Interrupts

- If the ADC interrupt is not required, then this step can be skipped. Otherwise, the ADC interrupt should be enabled from the NVIC tab in the configuration menu as in Fig. 7.11. The interrupt priority can be changed under the “System Core - NVIC” section. To note here, it is suggested to lower the ADC interrupt priority since the HAL library functions use the SysTick interrupt to create precise 1 ms ticks for internal operations and the HAL_Delay function. The SysTick interrupt has the highest priority among peripheral units. Hence, assigning a lower priority to the ADC interrupt will not affect it. Otherwise, precision of the HAL_Delay function may degrade.

Setting Up Triggers

- The trigger source can be selected from the “External Trigger Conversion Source” option under the “ADC_Regular_ConversionMode” setting in the configuration menu as in Fig. 7.11. The default value for this setup is “Regular Conversion.” If a hardware trigger will be used, then one of the EXTI line 11 or timer source with capture/compare or trigger out event options should be selected. To select the EXTI line 11, the 11th pin of any port should be configured as “ADCx_EXTI11” on the “Device Configuration Tool” window first. The trigger edge should also be selected as rising, falling, or rising and falling edge from the “External Trigger Conversion Edge” option under the configuration menu as in Fig. 7.11.

Setting Up Clock Options

- The last step is configuring the clock for the ADC module. Here, we assume that the clock configuration for the STM32F4 microcontroller has been done.
- We can set the ADC clock divider as 2, 4, 6, or 8 from the clock prescaler option in the configuration menu as in Fig. 7.11.
- The clock configuration of the microcontroller may not be suitable for the ADC module. For such cases, click on the “Clock Configuration” tab on the top menu of the “Device Configuration Tool” window. Then, click on the “Resolve Clock Issues” button to solve the problem. The reader can also set the ADC clock and its source clocks manually to the desired value in the same window.

ADC Setup via C++ Language

- We can benefit from Mbed and available functions there to adjust the ADC module properties in C++ language. To do so, we should first adjust ADC channels. This can be done by the class AnalogIn. Usage of this class is similar to the DigitalIn class introduced in Sect. 4.4.2. Therefore, we should supply a channel name to the AnalogIn class. Optionally, a Vref value can be provided to the class. If this value is not provided, then the AnalogIn class uses the default Vref source value.
- Mbed does not support different ADC configurations. Therefore, we cannot use the interrupt, hardware trigger, and injected mode options in it. In fact, we can only use the ADC module in its default configuration which is the single-channel, single-shot mode regular conversion with software trigger. We can indirectly realize hardware trigger using other STM32F4 microcontroller peripheral units. As an example, we can trigger the ADC module in a timer or GPIO callback function to realize the hardware trigger. However, this mode should be used with care as the ADC function used in the callback function blocks the code. Hence, it is better to set a flag in the callback function and realize the ADC operation in the main function based on the flag value.

ADC Usage in C++ Language

- …next week

Timer example 1

Listing 6.6 Toggling the green and red LEDs using timer interrupts, in C++ language

```
#include "mbed.h"

DigitalOut greenLED(LED1);
DigitalOut redLED(LED2);
Timeout timeoutDelay;
Ticker ticker1;
Ticker ticker2;

void toggleGreenLED()
{
greenLED = !greenLED;
}

void toggleRedLED()
{
redLED = !redLED;
}

void delay05()
{
toggleRedLED();
ticker1.attach(&toggleRedLED, 1s);
}

int main()
{
timeoutDelay.attach(&delay05, 500ms);
ticker2.attach(&toggleGreenLED, 1s);

while (1);
}
```

Timer example 2

Listing 6.7 Measuring button pressing time using GPIO interrupts and timer, in C++ language

```
#include "mbed.h"

InterruptIn userButton(BUTTON1);
DigitalOut greenLED(LED1);
Timer timer1;

uint32_t elapsedMs;

void triggerRise()
{
timer1.start();
}

void triggerFall()
{
timer1.stop();
elapsedMs = chrono::duration_cast<chrono::milliseconds>(timer1.
    elapsed_time()).count();
if (elapsedMs > 3000)
greenLED = 1;
else
greenLED = 0;
timer1.reset();
}

int main()
{
userButton.rise(&triggerRise);
userButton.fall(&triggerFall);

while (1);
}
```

Timer example 3

Listing 6.8 Using the watchdog timer to reset stuck system, in C++ language

```
#include "mbed.h"

DigitalOut greenLED(LED1);
InterruptIn userButton(BUTTON1);
Ticker ticker1;

void toggleGreenLED()
{
greenLED = !greenLED;
Watchdog::get_instance().kick();
}

void trigger()
{
while (1);
}

int main()
{
Watchdog &wdt = Watchdog::get_instance();
ticker1.attach(&toggleGreenLED, 1s);
wdt.start(4000);

userButton.rise(&trigger);

while (1);
}
```

Lab task, due to may 16

- Use mbed
- 2 LEDs and 1 pushbuttons will be used.
    - If user button is pressed more than 3 sec, Led 1 toggles.
    - If user button is pressed less than 3 sec, Led 2 toggles.
- This procedure goes indefinitely.